

10月14日作业

厉帅 10235501418

复习题

2. pagerank的主要设计思想涉及链接分析、随机游走模型和迭代计算。
其中链接分析又分为重要性假设和数量与质量并重；
随机游走模型包括马尔可夫链和平稳分布；
迭代计算需要初始化、迭代更新、阻尼因子和收敛
3. 贝叶斯定理的内容是 $P(A|B)=P(B|A)*P(A)/P(B)$
应用：在机器学习中，贝叶斯定理被广泛应用于分类问题；在决策分析中，贝叶斯原理是一种重要的决策原则，用于计算在给定条件下采取某种行动的最佳选择。
4. 蒙特卡洛方法的基本原理是通过数字模拟试验，得到所要求解的出现某种事件的概率，作为问题的近似解。
5. 梯度下降法的主要思想是通过不断迭代沿着函数的梯度的反方向更新参数，以逐渐接近函数的最小值。

练习题

1.

```
import numpy as np

# 生成100个服从标准正态分布的样本
samples = np.random.randn(100)

print(samples)
```

2.

```
import numpy as np
import matplotlib.pyplot as plt
samples = np.random.randn(100) #100个样本
plt.hist(samples, bins=30, edgecolor='black', alpha=0.7, density=True)
from scipy.stats import norm
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = norm.pdf(x, 0, 1)
plt.plot(x, p, 'k', linewidth=2)
title = "Fit results: mean = %.2f, std = %.2f" % (np.mean(samples),
np.std(samples))
plt.title(title)
plt.show()
```

3.

```
import numpy as np
A = np.array([[2, 1],
              [4, 5]])
eigenvalues, eigenvectors = np.linalg.eig(A)
print("特征值:")
print(eigenvalues)
print("特征向量:")
print(eigenvectors)
```

5.

```
import numpy as np
X = np.array([1, -1, 4])
Y = np.array([2, 1, 3])
Z = np.array([1, 3, -1])
data_matrix = np.column_stack((X, Y, Z))
covariance_matrix = np.cov(data_matrix, rowvar=False)
print("协方差矩阵:")
print(covariance_matrix)
```

```
import numpy as np
import matplotlib.pyplot as plt
def f(x):
    return 0.25 * (x - 0.5)**2 + 1
def grad_f(x):
    return 0.25 * 2 * (x - 0.5)
def gradient_descent(initial_x, learning_rate, max_iterations):
    x = initial_x
    x_values = [x]
    f_values = [f(x)]
    for i in range(max_iterations):
        grad = grad_f(x)
        x = x - learning_rate * grad
        x_values.append(x)
        f_values.append(f(x))
    return x_values, f_values
initial_x = 2.0
learning_rate = 0.1
max_iterations = 50
x_values, f_values = gradient_descent(initial_x, learning_rate, max_iterations)
x_range = np.linspace(-1, 2, 400)
f_range = f(x_range)
plt.plot(x_range, f_range, label='f(x) = 0.25*(x-0.5)^2 + 1')
plt.plot(x_values, f_values, 'ro-', label='Gradient Descent Path', markersize=2)
plt.scatter(x_values[-1], f_values[-1], color='r', zorder=5)
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.title('Gradient Descent for Local Minimum')
plt.grid(True)
plt.show()
```

